

Name : Vivek Chaudhari

Project 6:

WebTierX: Multi tier web app with load balancing

Objective

Deploy a scalable 3-tier web application using AWS services with proper separation of frontend, backend, and database

User → Load Balancer → Frontend EC2 (Nginx)

↓

Backend EC2 (Flask API)

↓

RDS MySQL Database

Step 1: Launch Backend EC2 Instance

- Go to AWS EC2 → Launch Instance
- Choose Ubuntu
- Configure Security Group:
 - SSH (22)
 - Custom TCP (5000)
- Connect to EC2 terminal

◆ Step 2: Setup Backend Environment

Install required packages:

```
sudo apt update
```

```
sudo apt install python3-pip -y
```

Install Python libraries:

```
pip3 install flask flask-cors --break-system-packages
```

◆ Step 3: Create Backend Application

Create file:

```
nano app.py
```

Add code:

```
from flask import Flask
```

```
from flask_cors import CORS
```

```
app = Flask(__name__)
```

```
CORS(app)
```

```
@app.route('/')
def home():
    return "Backend Running!"

@app.route('/db')
def db():
    return "Database Connected Successfully!"

app.run(host='0.0.0.0', port=5000)

Run backend:

python3 app.py
```

◆ Step 4: Create RDS Database

- Go to RDS → Create database
- Engine: MySQL
- Instance type: db.t3.micro
- Set username & password

Configure:

- Enable public access (for testing)
 - Allow port 3306 in security group
-

◆ Step 5: Launch Frontend EC2 Instance

- Launch another EC2 instance (Ubuntu)
 - Configure Security Group:
 - SSH (22)
 - HTTP (80)
-

◆ Step 6: Setup Frontend (Nginx)

Install Nginx:

```
sudo apt update
sudo apt install nginx -y
```

Edit HTML file:

```
sudo nano /var/www/html/index.html
```

Add frontend code (fetch /db endpoint).

Restart server:

```
sudo systemctl restart nginx
```

◆ Step 7: Create Load Balancer

- Go to EC2 → Load Balancers
- Create **Application Load Balancer**

Configure:

- Listener: HTTP (80)
- Scheme: Internet-facing
- Select VPC and subnets

◆ Step 8: Create Target Groups

Frontend Target Group

- Port: 80
- Add frontend EC2

Backend Target Group

- Port: 5000
- Add backend EC2

Health Check:

- Path: /

◆ Step 9: Configure Listener Rules

Go to Load Balancer → Listeners → Rules

Add:

- /db* → Forward to backend-target
- Default → Forward to frontend-target

◆ Step 10: Fix Security Groups

Backend EC2:

- Allow port 5000 from Load Balancer

Load Balancer:

- Allow port 80 from internet

◆ Step 11: Test Application

Open:

http://<load-balancer-dns>

- Click button → calls backend
- Backend → connects to database

✓ Final Output

- Frontend hosted on EC2
- Backend API running on EC2
- Database hosted on RDS
- Load Balancer routing traffic

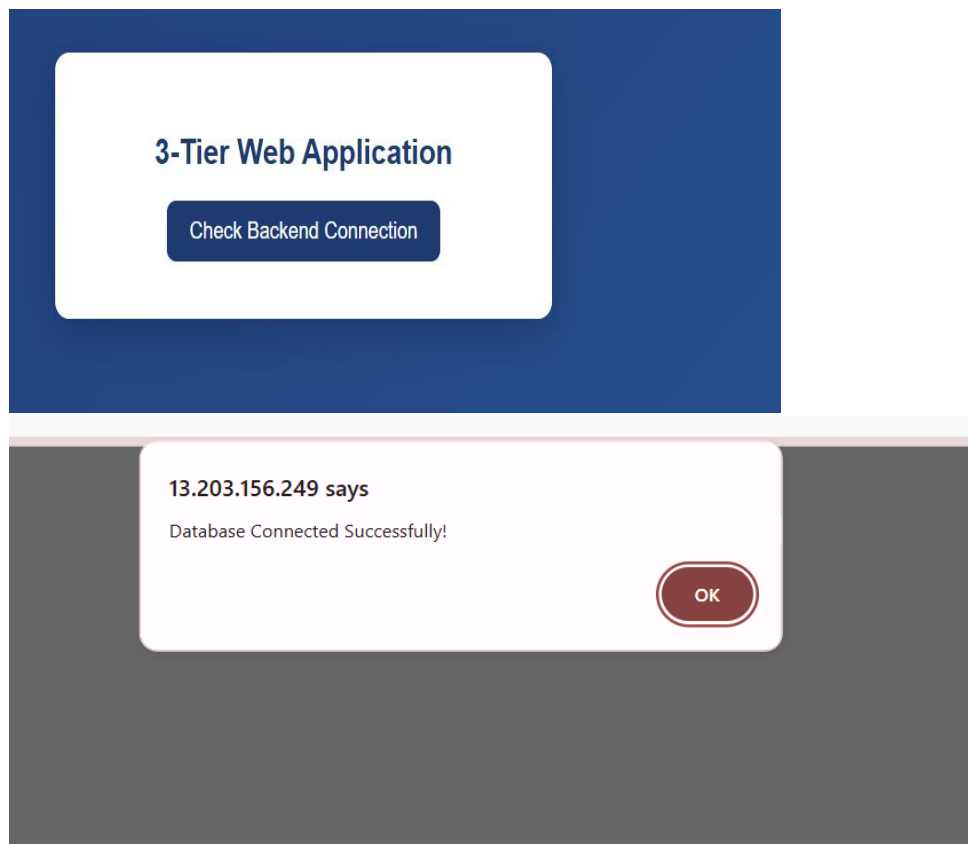
Technologies Used

- AWS EC2
- AWS RDS
- AWS Load Balancer
- Python (Flask)
- Nginx
- MySQL

```
03.186.19.237 - - [23/Apr/2026 14:35:38] "GET /favicon.ico HTTP/1.1" 404 -
03.186.19.237 - - [23/Apr/2026 14:35:55] "GET /db HTTP/1.1" 200 -
03.186.19.237 - - [23/Apr/2026 14:35:55] "GET /favicon.ico HTTP/1.1" 404 -
03.186.19.237 - - [23/Apr/2026 14:36:02] "GET /db HTTP/1.1" 200 -
03.186.19.237 - - [23/Apr/2026 14:36:08] "GET /db HTTP/1.1" 200 -
03.186.19.237 - - [23/Apr/2026 14:36:34] "GET /db HTTP/1.1" 200 -
(mysenv) ubuntu@ip-172-31-2-21:~$ nano app.py
(mysenv) ubuntu@ip-172-31-2-21:~$ pip install flask-cors
Collecting flask-cors
  Downloading flask_cors-6.0.2-py3-none-any.whl.metadata (5.3 kB)
Requirement already satisfied: flask>=0.9 in ./mysenv/lib/python3.12/site-packages (6.0.2)
Requirement already satisfied: Werkzeug>=0.7 in ./mysenv/lib/python3.12/site-packages (3.0.2)
Requirement already satisfied: blinker>=1.9.0 in ./mysenv/lib/python3.12/site-packages (1.9.0)
Requirement already satisfied: click>=8.1.3 in ./mysenv/lib/python3.12/site-packages (8.1.3)
Requirement already satisfied: itsdangerous>=2.2.0 in ./mysenv/lib/python3.12/site-packages (2.2.0)
Requirement already satisfied: Jinja2>=3.1.2 in ./mysenv/lib/python3.12/site-packages (3.1.2)
Requirement already satisfied: MarkupSafe>=2.1.1 in ./mysenv/lib/python3.12/site-packages (2.1.1)
Downloading flask_cors-6.0.2-py3-none-any.whl (13 kB)
Installing collected packages: flask-cors
Successfully installed flask-cors-6.0.2
(mysenv) ubuntu@ip-172-31-2-21:~$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.31.2.21:5000
Press CTRL+C to quit
03.186.19.237 - - [23/Apr/2026 14:39:32] "GET /db HTTP/1.1" 200 -
03.186.19.237 - - [23/Apr/2026 14:39:33] "GET /favicon.ico HTTP/1.1" 404 -
03.186.19.237 - - [23/Apr/2026 14:39:47] "GET /db HTTP/1.1" 200 -
03.186.19.237 - - [23/Apr/2026 14:40:15] "GET /db HTTP/1.1" 200 -
03.186.19.237 - - [23/Apr/2026 14:40:36] "GET /db HTTP/1.1" 200 -
03.186.19.237 - - [23/Apr/2026 14:40:53] "GET /db HTTP/1.1" 200 -
```

```
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-1-67:~$ sudo nano /var/www/html/index.html
ubuntu@ip-172-31-1-67:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-04-23 14:21:52 UTC; 6min ago
     Docs: man:nginx(8)
    Process: 1949 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 1951 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 1979 (nginx)
      Tasks: 2 (limit: 1117)
     Memory: 1.7M (peak: 3.7M)
        CPU: 18ms
   CGroup: /system.slice/nginx.service
           └─1979 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─1982 "nginx: worker process"

Apr 23 14:21:52 ip-172-31-1-67 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Apr 23 14:21:52 ip-172-31-1-67 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-1-67:~$ sudo systemctl restart nginx
ubuntu@ip-172-31-1-67:~$ sudo nano /var/www/html/index.html
ubuntu@ip-172-31-1-67:~$ sudo systemctl restart nginx
ubuntu@ip-172-31-1-67:~$ sudo nano /var/www/html/index.html
ubuntu@ip-172-31-1-67:~$ sudo systemctl restart nginx
```



This project implements a **3-tier web application on AWS** using EC2, RDS, and an Application Load Balancer. The frontend is hosted on EC2 with Nginx, the backend runs a Flask API on another EC2 instance, and the database is managed using RDS MySQL. The Load Balancer routes user requests efficiently, enabling communication between all layers.